LAPORAN PRAKTIKUM

JOBSHEET 10 - AUTENTIKASI DAN MANAJEMEN SESI PENGGUNA



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Perancangan Form Login dan Registrasi Akun Petugas 3

2.1.1. 1.1 Struktur Form Login dan Form Registrasi 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 5

2.2. Percobaan 2: Implementasi Middleware Guard Sesi (includes/auth.php) 6

2.2.1. 2.1 Mekanisme Redirection pada Guard Sesi 6

2.2.2. Langkah-langkah 7

2.2.3. Pertanyaan 8

2.3. Percobaan 3: Verifikasi Login dan Dashboard Terproteksi 9

2.3.1. 3.1 Verifikasi Kredensial dan Inisialisasi Sesi 9

2.3.2. Langkah-langkah 10

2.3.3. Pertanyaan 11

2.4. Percobaan 4: Siklus Logout dan Penghancuran Sesi 12

2.4.1. 4.1 Terminasi Sesi Pengguna 12

2.4.2. Langkah-langkah 12

2.4.3. Pertanyaan 14

3. Tugas 15

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 10 15

1. Tujuan

- Memahami konsep dasar dan perbedaan mendasar arsitektur Autentikasi vs Otorisasi pada aplikasi web berbasis PHP dan PostgreSQL.
- Mengimplementasikan skema tabel users untuk penyimpanan kredensial akun pengguna/petugas perpustakaan secara aman.
- Menerapkan teknik hashing kata sandi satu arah yang aman menggunakan algoritma BCrypt melalui fungsi bawaan PHP password_hash().
- Menerapkan verifikasi kredensial saat autentikasi masuk menggunakan password_verify() tanpa membandingkan plain text.
- Menguasai siklus hidup sesi HTTP (session_start, $_SESSION, dan session_destroy) serta mitigasi serangan Session Fixation via session_regenerate_id(true).
- Membangun berkas middleware/guard includes/auth.php untuk memproteksi halaman internal dari akses tak terotorisasi.
- Mengimplementasikan tata letak antarmuka navigasi dinamis (header.php) yang menampilkan identitas pengguna dan tombol logout.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

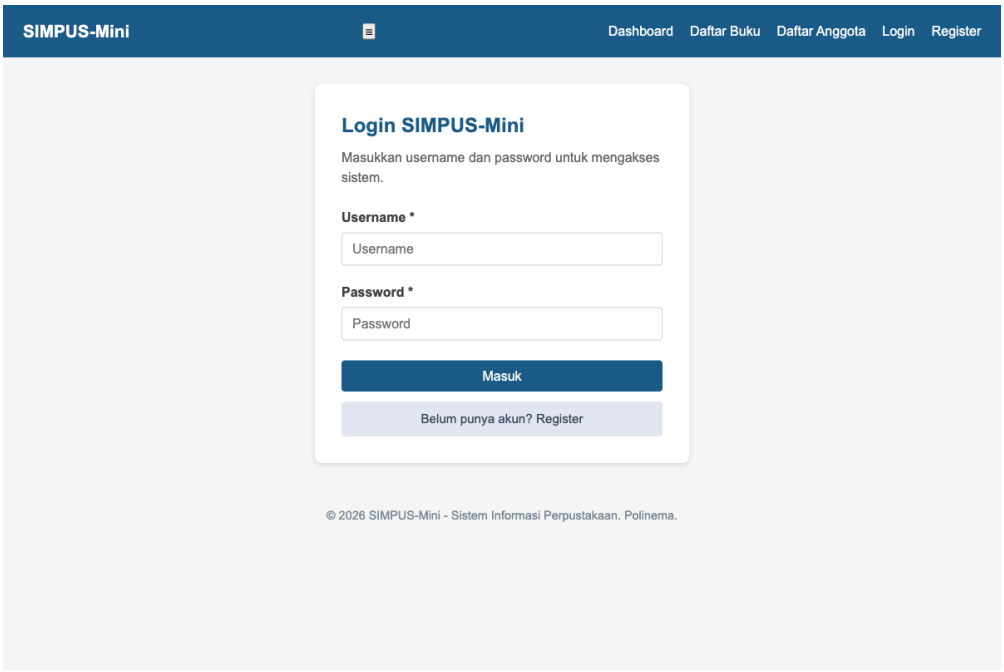
2.1. Percobaan 1: Perancangan Form Login dan Registrasi Akun Petugas

Pada percobaan pertama, antarmuka autentikasi dirancang untuk menangani pendaftaran petugas baru (`register.php`) dan proses masuk pengguna terdaftar (`login.php`).

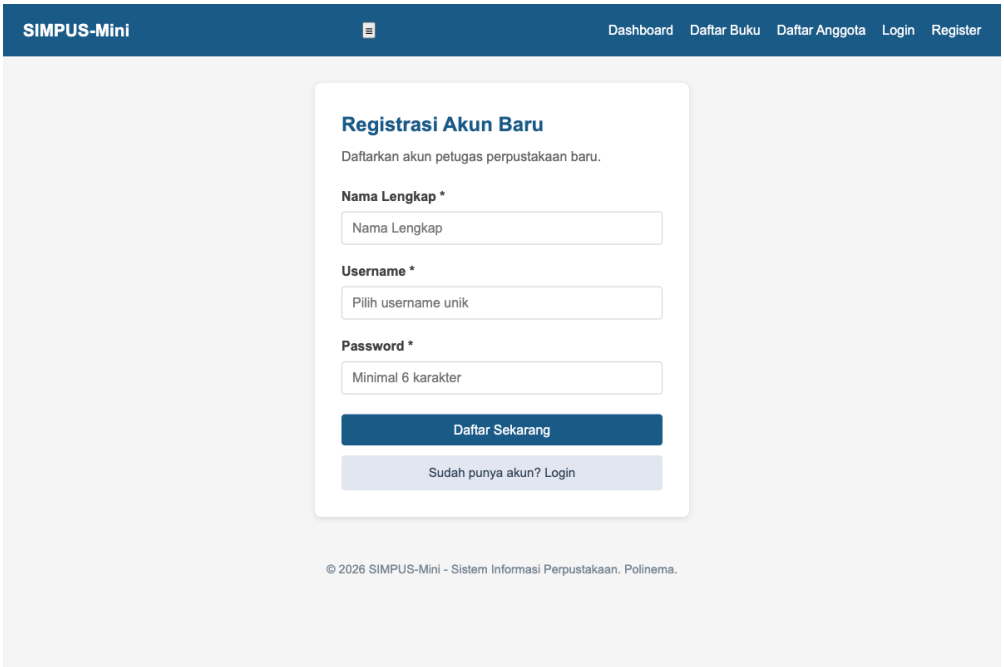
2.1.1. 1.1 Struktur Form Login dan Form Registrasi

Formulir pendaftaran mengumpulkan atribut Nama Lengkap, Username unik, dan Password dengan batasan keamanan panjang minimal 6 karakter. Sedangkan form login menerima pasangan kredensial username dan password:

```
<form action="proses_register.php" method="POST">
    <input type="text" id="nama" name="nama" required placeholder="Nama Lengkap">
    <input type="text" id="username" name="username" required placeholder="Pilih
username unik">
    <input type="password" id="password" name="password" required minlength="6">
    <button type="submit" class="btn btn-primary">Daftar Sekarang</button>
</form>
```



Gambar 1.1: Antarmuka Halaman Login Petugas SIMPUS-Mini.



Gambar 1.2: Antarmuka Halaman Registrasi Akun Petugas Baru.

2.1.2. Langkah-langkah

1. Membuat skrip antarmuka `auth/login.php` dan `auth/register.php`.
2. Menyematkan elemen form dengan metode HTTP POST dan atribut pengaman input.
3. Mengintegrasikan tautan navigasi bolak-balik antara login dan registrasi.
4. Menguji tampilan antarmuka form login dan registrasi pada browser.

2.1.3. Pertanyaan

1. Mengapa kata sandi wajib di-hash menggunakan algoritma BCrypt (`password_hash()`) dan dilarang menggunakan algoritma cepat seperti MD5 atau SHA-1?

    - Algoritma MD5 dan SHA-1 dirancang untuk perhitungan integritas data berkecepatan tinggi, sehingga sangat rentan terhadap serangan brute-force modern via GPU serta serangan kamus tabel pelangi (**Rainbow Table**). Sebaliknya, BCrypt dirancang memiliki faktor biaya komputasi adaptif (**work factor**) yang sengaja memperlambat proses brute-force penyerang secara eksponensial, serta otomatis menyertakan **cryptographic salt** unik pada setiap hasil hash.

2.2. Percobaan 2: Implementasi Middleware Guard Sesi (includes/auth.php)

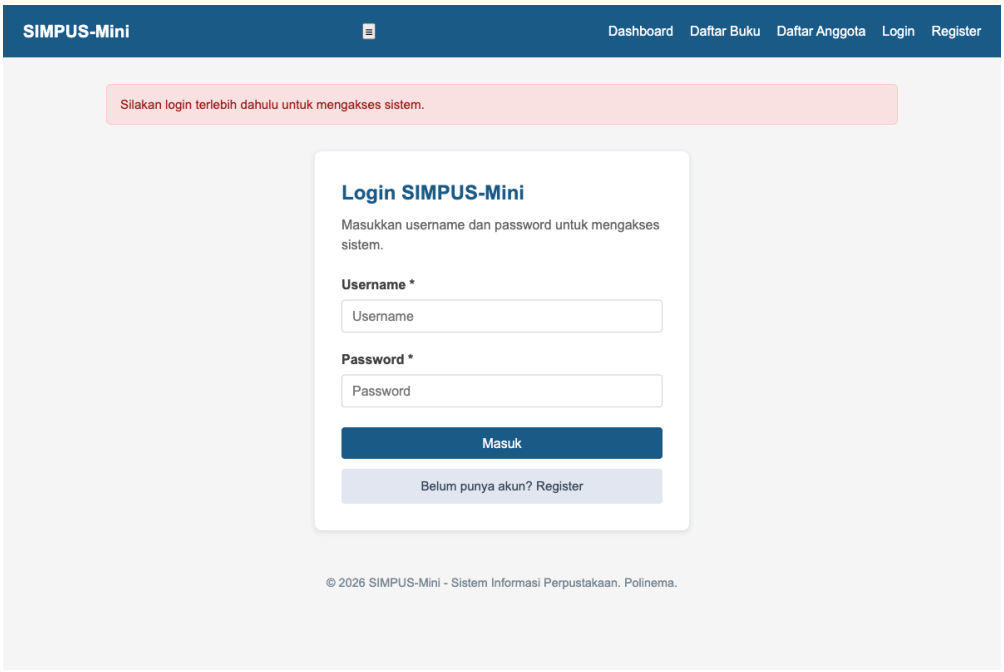
Percobaan kedua mengimplementasikan gerbang pengaman akses (**session guard**) pada berkas `includes/auth.php`. Guard ini bertugas memeriksa keberadaan sesi login aktif sebelum mengizinkan pemrosesan halaman terproteksi.

2.2.1. 2.1 Mekanisme Redirection pada Guard Sesi

Bila sesi pengguna tidak ditemukan di variabel `$_SESSION['user']`, pengunjung langsung dialihkan ke halaman login dengan menyertakan pesan peringatan flash:

```
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

if (!isset($_SESSION['user'])) {
    $_SESSION['flash'] = [
        'type' => 'error',
        'pesan' => 'Silakan login terlebih dahulu untuk mengakses sistem.'
    ];
    $current_folder = basename(dirname($_SERVER['SCRIPT_NAME']));
    $root = ($current_folder === 'books' || $current_folder === 'members') ? '..' :
'.';
    header("Location: $root/auth/login.php");
    exit;
}
```



Gambar 1.3: Pengalihan Otomatis Menuju Halaman Login Saat Pengunjung Mencoba Mengakses Halaman Terproteksi Tanpa Sesi Aktif.

2.2.2. Langkah-langkah

1. Membuat berkas guard `includes/auth.php` dengan pengecekan `isset($_SESSION['user'])`.
2. Menambahkan pengaturan header HTTP 302 redirect menuju `auth/login.php`.
3. Menyematkan perintah `require_once auth.php` pada baris pertama di berkas `index.php`, `books/*.php`, dan `members/*.php`.
4. Menguji akses langsung ke URL terproteksi melalui peramban web dan mengamati respons pengalihan.

2.2.3. Pertanyaan

1. Mengapa berkas guard `includes/auth.php` harus di-require sebelum berkas `includes/header.php` menghasilkan keluaran HTML?

    - Sesuai spesifikasi protokol HTTP, blok header respons harus dikirimkan sebelum badan (**body**) respons HTML pertama dipancarkan. Fungsi `header("Location: ...")` di PHP bertugas menuliskan header HTTP tersebut. Apabila `header.php` telah menghasilkan output HTML, PHP akan memicu status **headers already sent** dan membatalkan instruksi pengalihan, sehingga halaman terproteksi berisiko bocor ke pengguna tak sah.

2.3. Percobaan 3: Verifikasi Login dan Dashboard Terproteksi

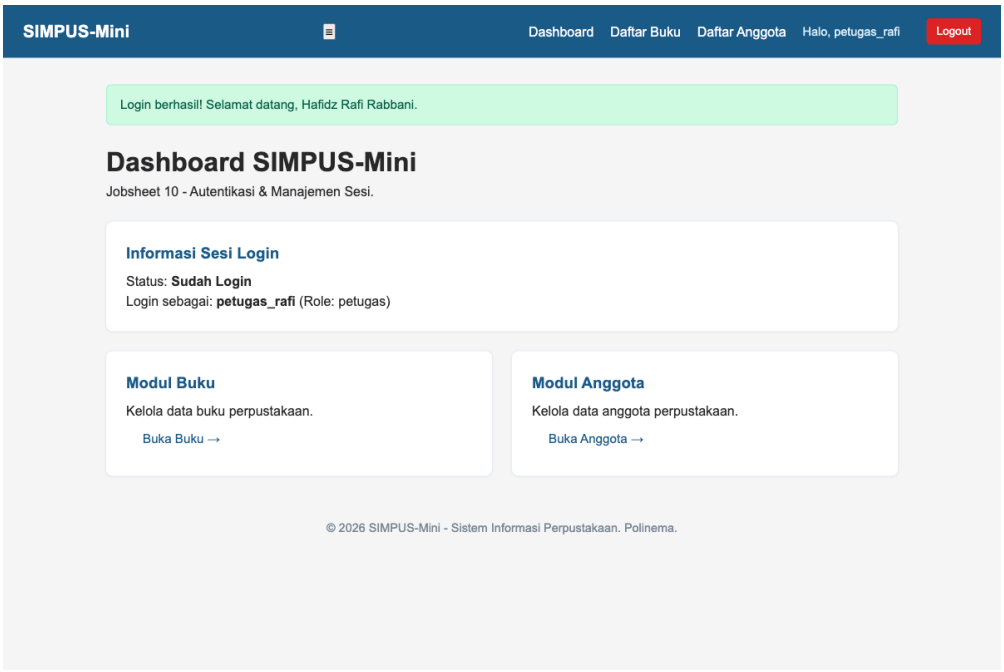
Percobaan ketiga memvalidasi kredensial pengguna pada `proses_login.php` menggunakan `password_verify()`, meregenerasi ID sesi untuk mencegah **Session Fixation**, serta menampilkan dasbor utama yang terproteksi.

2.3.1. 3.1 Verifikasi Kredensial dan Inisialisasi Sesi

Kueri pencarian pengguna dijalankan, lalu kecocokan kata sandi diverifikasi secara aman:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
$stmt->execute([':username' => $username]);
$user = $stmt->fetch();

if ($user && password_verify($password, $user['password'])) {
    session_regenerate_id(true);
    $_SESSION['user'] = [
        'id' => $user['id'],
        'nama' => $user['nama'],
        'username' => $user['username'],
        'role' => $user['role'] ?? 'petugas',
    ];
    header('Location: ../index.php');
    exit;
}
```



Gambar 1.4: Tampilan Dasbor Utama Terproteksi Menampilkan Status Login Petugas dan Navigasi Adaptif.

2.3.2. Langkah-langkah

1. Mengembangkan skrip `auth/proses_login.php` dengan pencocokan `password_verify()`.
2. Menyematkan pemanggilan `session_regenerate_id(true)` saat kredensial valid.
3. Memperbarui berkas `includes/header.php` agar menampilkan salam pengguna dan tombol logout secara dinamis.
4. Menguji login dengan akun petugas dan memverifikasi tampilan dashboard.

2.3.3. Pertanyaan

1. Apa fungsi pemanggilan `session_regenerate_id(true)` saat proses login berhasil?
    - `session_regenerate_id(true)` memperbarui ID sesi aktif dengan token acak baru sekaligus menghapus rekaman sesi lama di server. Tindakan ini memitigasi ancaman keamanan **Session Fixation**, di mana penyerang mencoba memaksakan ID sesi tertentu kepada korban sebelum login.

2.4. Percobaan 4: Siklus Logout dan Penghancuran Sesi

Percobaan keempat mengimplementasikan mekanisme keluar sistem pada berkas `auth/logout.php`. Seluruh data sesi dibersihkan, cookie sesi dihapus, dan sesi dihancurkan secara permanen di sisi server.

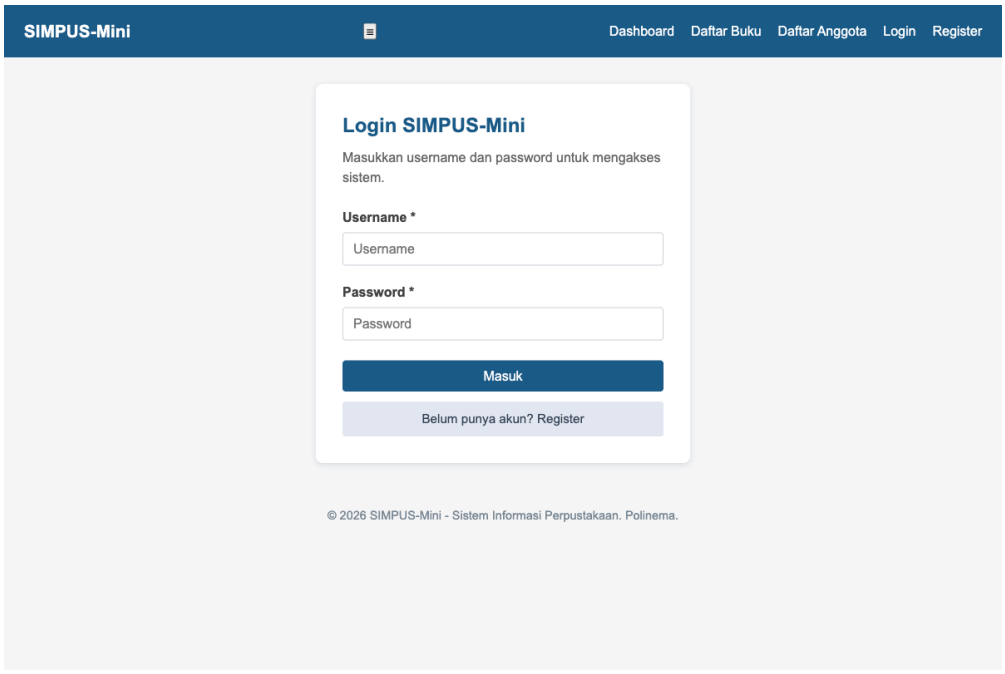
2.4.1. 4.1 Terminasi Sesi Pengguna

Pembersihan menyeluruh dilakukan untuk menjamin sesi tidak dapat digunakan kembali (**session replay**):

```
session_start();
$_SESSION = [];

if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

session_destroy();
```



Gambar 1.5: Notifikasi Sukses Logout dan Pengalihan Kembali Menuju Form Login.

2.4.2. Langkah-langkah

1. Membuat berkas pengendali `auth/logout.php`.
2. Mengosongkan array `$_SESSION = []` dan menghapus cookie sesi klien.

3. Memanggil fungsi `session_destroy()` untuk mematikan sesi di server.
4. Menguji tombol Logout dari navbar dan memastikan sesi telah berakhir sepenuhnya.

2.4.3. Pertanyaan

1. Mengapa pengosongan variabel `$_SESSION = []` saja tidak cukup untuk logout tanpa memanggil `session_destroy()`?

    - Mengosongkan `$_SESSION` hanya menghapus data sesi yang dimuat pada memori skrip PHP saat itu, namun berkas atau rekaman sesi dengan ID bersangkutan masih tetap tersimpan di media penyimpanan sesi server. Pemanggilan `session_destroy()` mutlak diperlukan untuk menghapus berkas penyimpanan sesi tersebut dari disk server sehingga ID sesi tersebut mati secara permanen.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 10

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 10:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Jelaskan perbedaan mendasar antara **Authentication** (Autentikasi) dan **Authorization** (Otorisasi) dalam arsitektur aplikasi web! Bagaimana keduanya diimplementasikan pada SIMPUS-Mini di Jobsheet 10? **Jawaban:** - **Autentikasi (Authentication)**: Adalah proses verifikasi identitas pengguna untuk membuktikan **siapa Anda** (misalnya pembuktian keabsahan pasangan username dan password). Pada SIMPUS-Mini, proses ini diimplementasikan melalui formulir `login.php` dan skrip `proses_login.php` menggunakan fungsi `password_verify()` terhadap tabel `users`. - **Otorisasi (Authorization)**: Adalah proses verifikasi hak akses untuk menentukan **apa yang boleh Anda lakukan** setelah identitas terbukti sah (misalnya wewenang role `admin` vs `petugas`, atau akses tamu vs pengguna terdaftar). Pada SIMPUS-Mini, otorisasi awal diterapkan via middleware `includes/auth.php` yang membatasi akses ke menu pengelolaan data buku dan anggota hanya bagi pengunjung yang telah memiliki sesi login yang valid (`$_SESSION['user']`).
2	**Soal:** Mengapa kata sandi pengguna tidak boleh disimpan dalam bentuk teks polos (**plain text**) di basis data? Jelaskan mekanisme fungsi `password_hash()` dan `password_verify()` dalam PHP! **Jawaban:** Menyimpan kata sandi dalam bentuk **plain text** merupakan celah keamanan fatal: jika terjadi kebocoran basis data (**data breach** atau

No	Pertanyaan dan Jawaban Konseptual
	SQL injection), seluruh akun pengguna langsung terekspos tanpa perlindungan, memicu kompromi massal (**credential stuffing**). - Fungsi `password_hash($password, PASSWORD_BCRYPT)` menghasilkan string hash kriptografis satu arah (**one-way hash**) yang menyertakan **salt** acak unik dan faktor komputasi adaptif (**cost factor**). Fungsi ini menjamin password asli tidak dapat direkonstruksi kembali. - Fungsi `password_verify($password, $hash)` menerima password mentah yang diketikkan pengguna saat login, mengambil parameter salt dan cost dari string hash database, menghash ulang input tersebut, dan membandingkannya dalam waktu konstan (**constant-time comparison**) untuk menghindari serangan **timing attack**.
3	**Soal:** Mengapa fungsi `session_status() === PHP_SESSION_NONE` perlu diperiksa sebelum memanggil `session_start()`? Apa akibatnya jika `session_start()` dipanggil berulang kali dalam satu siklus request? **Jawaban:** Pemeriksaan `session_status() === PHP_SESSION_NONE` menjamin bahwa sesi PHP hanya diinisialisasi tepat satu kali. Dalam arsitektur aplikasi modular, beberapa berkas terpisah (seperti `auth.php`, `header.php`, dan skrip halaman) kerap membutuhkan akses ke variabel sesi. Jika `session_start()` dipanggil saat sesi sudah aktif, PHP akan memunculkan peringatan (**Notice: session already active**). Peringatan ini dapat mengotori buffer keluaran (**output buffer**) dan berpotensi merusak pengiriman header HTTP pada arsitektur web.
4	**Soal:** Jelaskan bagaimana guard `includes/auth.php` mampu melindungi halaman internal bahkan saat koneksi basis data sedang terputus atau mengalami kegagalan! **Jawaban:** Guard `includes/auth.php` dirancang secara mandiri tanpa bergantung pada koneksi database: guard ini hanya mengevaluasi keberadaan

No	Pertanyaan dan Jawaban Konseptual
	array sesi `$_SESSION['user']` yang tersimpan pada penyimpanan sesi lokal web server. Ketika koneksi basis data gagal, pengunjung yang belum login tetap akan dievaluasi dan langsung dibelokkan (**HTTP 302 redirect**) ke halaman login sebelum kode program mencoba menyambung ke objek PDO. Hal ini memastikan lapisan keamanan sistem tetap kokoh dan tidak mengekspos galat sistem database kepada pihak luar.
5	**Soal:** Jelaskan konsep ancaman keamanan **Session Fixation** dan bagaimana fungsi `session_regenerate_id(true)` mencegah eksploitasi tersebut saat proses autentikasi berhasil! **Jawaban:** **Session Fixation** adalah teknik serangan di mana peretas menentukan terlebih dahulu ID sesi korban (misal menyematkannya via parameter URL atau tautan jebakan). Jika korban berhasil login tanpa adanya pergantian ID sesi, peretas dapat langsung mengambil alih akun korban karena peretas telah memegang ID sesi tersebut. Fungsi `session_regenerate_id(true)` mengganti nilai ID sesi lama dengan ID sesi baru yang acak seketika setelah login diverifikasi, serta menghapus data sesi lama di server (`delete_old_session = true`). Hal ini membatalkan token yang dipegang peretas dan menutup celah pengambilalihan akun secara total.